Design Analysis Report: Smart Waste Collection Management System

1. Introduction

Purpose. This report analyses the design of the Smart Waste Collection Management System (SWCMS) version 1.0, as specified in the team's SRS dated 14-Sep-2026. The SRS defines what the system must do. This report shows how the design delivers it, using eight views: use case, class, architecture, activity, sequence, collaboration, state transition and component. It also explains where open decisions remain.

Scope. The report covers the whole system: ultrasonic sensor nodes on bins, the cloud backend, the admin web dashboard, and the driver and citizen mobile apps. Hardware wiring and UI screen design are out of scope. The use case and class diagrams (sections 3 and 5) define the actor roles and the domain model that the other views build on.

References. SWCMS Software Requirements Specification v1.0; IEEE Std 830-1998; IEEE SRS template (Wiegers); Lab 1 Problem Identification and Solution document.

2. Design goals and drivers

The design aims to cut unnecessary collection trips and prevent bin overflow, while staying cheap enough for an academic prototype. Six constraints from the SRS shape most of the choices.

Driver	SRS source	Design response
Low-power outdoor sensors	2.5	Sensor reports every 15 minutes, not continuously
Weak or intermittent GPRS	2.5, 3.4	Small HTTP/MQTT payloads; retry and log on failure
Low-cost APIs	2.5	Maps adapter allows OpenStreetMap or Google Maps
Route in under 10 s for 200 bins	REQ-RO-2	Dedicated Routing module with a heuristic solver
500 concurrent sensor nodes	5.1	Separate Ingestion service that can scale on its own
Easy maintenance and growth	5.4	Modular backend; one module per feature area

The design follows three principles: keep the sensor simple and push logic to the server, keep each backend module single-purpose, and hide third-party services behind adapters.

3. Use case model

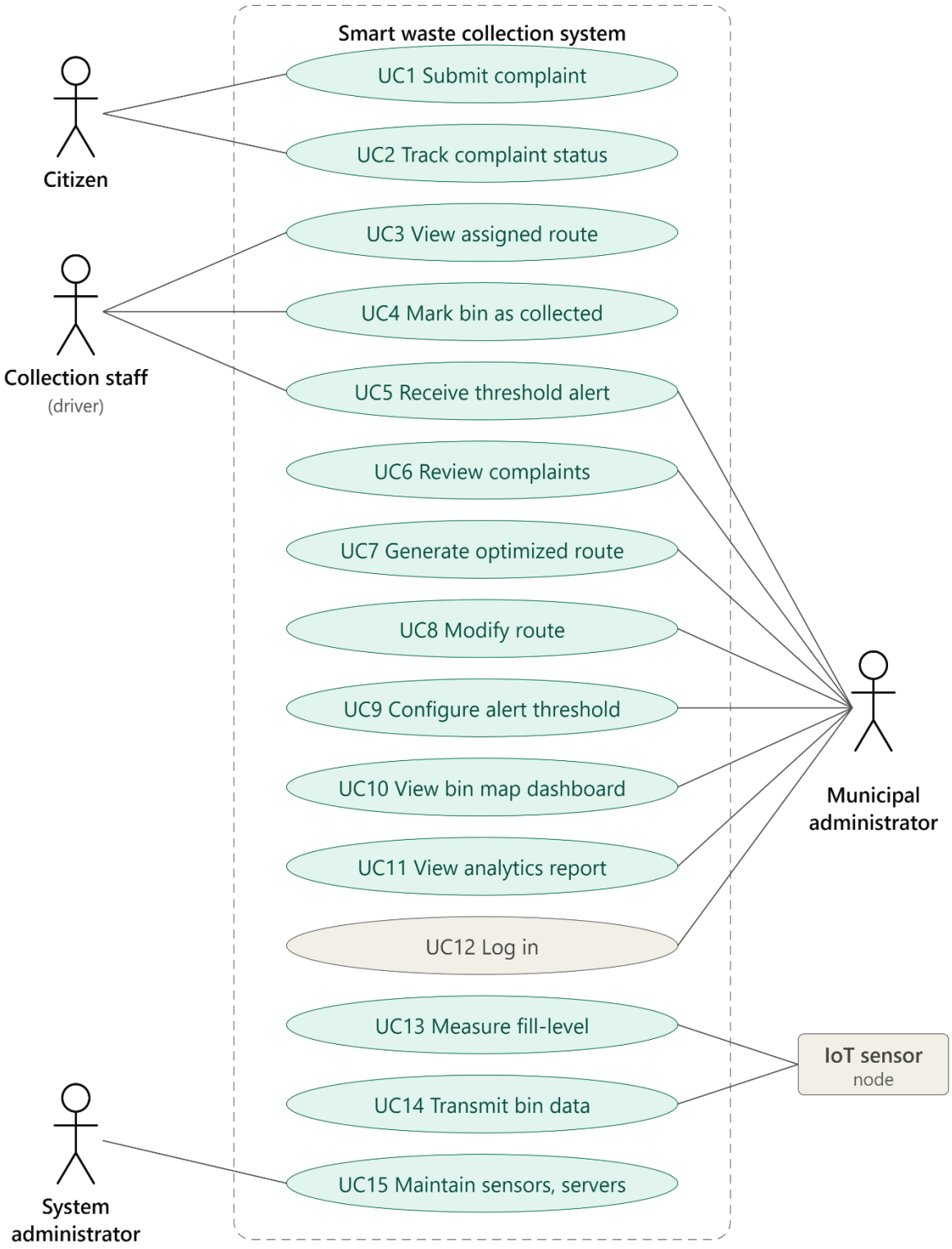
The use case model identifies who uses the system and what each actor needs from it. It has five actors and fifteen use cases, drawn from SRS sections 2.3, 3.1 and 4.

3.1 Actors and their roles

Actor	Role in the system
Citizen	Reports overflowing bins or requests an unscheduled pickup through the app or web form, and tracks the status of the complaint. Uses a tracking ID and needs no account.
Collection staff / driver	Receives the day's optimized route and bin alerts on the mobile app and marks bins as collected after pickup.
Municipal administrator	Controls the system from the web dashboard: sets thresholds, generates and adjusts routes, monitors bins, reviews complaints and reads analytics.
System administrator	Technical maintainer who keeps servers, sensors and connectivity running.
IoT sensor node	Secondary (non-human) actor. The ultrasonic sensor and microcontroller on each bin measure fill-level and send it to the backend.

3.2 Use case diagram

Five actors reach fifteen use cases. Dashed arrows mark the include and extend relationships described in section 3.4.


Smart waste collection system
UC1 Submit complaint
UC2 Track complaint status
UC3 View assigned route
UC4 Mark bin as collected
UC5 Receive threshold alert
UC6 Review complaints
UC7 Generate optimized route
UC8 Modify route
UC9 Configure alert threshold
UC10 View bin map dashboard
UC11 View analytics report
UC12 Log in
UC13 Measure fill-level
UC14 Transmit bin data
UC15 Maintain sensors, servers
Citizen
Collection staff
(driver)
Municipal administrator
System administrator
IoT sensor
node

3.3 Use case specifications

ID / Use case	Actor	Function	Precondition	Postcondition
UC1 Submit complaint	Citizen	Report an overflowing bin or request a pickup, with auto-detected location and an optional photo (REQ-CF-1, CF-2).	App or web form is open; location is available or selectable.	Complaint is stored with a unique tracking ID and appears on the admin dashboard as Pending.
UC2 Track complaint status	Citizen	View whether a complaint is Pending or Resolved (REQ-CF-3).	Complaint exists and the citizen has its tracking ID.	Current status is displayed; no data changes.
UC3 View assigned route	Driver	See the day's optimized route as a list or map.	Driver is logged in and a route has been generated and pushed.	Route is displayed on the driver's device.
UC4 Mark bin as collected	Driver	Confirm that a bin on the route has been emptied.	Bin is on the driver's assigned route and has been physically emptied.	Collection is recorded on the route stop and the bin is reset. Linked complaints are auto-marked Resolved.
UC5 Receive threshold alert	Driver, Administrator	Get a push notification when a bin crosses its threshold (REQ-AL-2, AL-3).	Threshold is configured and the server has received a reading above it.	Alert is delivered within 5 minutes of the server receiving the reading and is logged with bin ID and timestamp.
UC6 Review complaints	Administrator	Look at citizen complaints forwarded to the dashboard and act on them.	Admin is authenticated and at least one complaint exists.	Complaint is reviewed and any follow-up pickup is arranged.
UC7 Generate optimized route	Administrator	Compute the shortest route over all bins at or above the collection threshold	Admin is authenticated; bin locations are loaded; at least one bin is	Route is shown on the dashboard and pushed to the driver app, within 10 seconds for up to

ID / Use case	Actor	Function	Precondition	Postcondition
		(REQ-RO-1, RO-2).	above threshold; the mapping API is reachable.	200 bins.
UC8 Modify route	Administrator	Manually add or remove a bin on a generated route (REQ-RO-3).	A route has already been generated (UC7).	Updated route is saved and pushed to the driver.
UC9 Configure alert threshold	Administrator	Set the alert percentage per bin or globally (REQ-AL-1).	Admin is authenticated.	New threshold is saved and applies to all later readings.
UC10 View bin map dashboard	Administrator	See all bins on a map, colour-coded green, yellow or red by fill-level (REQ-DB-1).	Admin is authenticated; sensors have reported data.	Live map is displayed and the admin can open a bin's history.
UC11 View analytics report	Administrator	See collection counts and average fill-level per bin over a date range (REQ-DB-2).	Admin is authenticated; historical data exists for the range.	Report is displayed; no data changes.
UC12 Log in	Administrator, Driver	Authenticate with username and password, with optional OTP for admins (REQ-DB-3).	Account exists.	A valid session is created, or access is denied.
UC13 Measure fill-level	IoT sensor node	Measure distance to the waste surface and convert it to a fill percentage at a set interval, default 15 minutes (REQ-BM-1, BM-3).	Sensor is powered (battery or solar) and mounted on the bin.	Fill percentage is ready to send. A failed read is retried and logged.
UC14 Transmit bin data	IoT sensor node	Send bin ID, fill-level and timestamp to the backend over Wi-Fi/GPRS using HTTP	UC13 has produced a reading; network	Backend stores the reading and updates the bin's dashboard status.

ID / Use case	Actor	Function	Precondition	Postcondition
		or MQTT (REQ-BM-2).	connectivity is available.	
UC15 Maintain sensors and servers	System administrator	Keep hardware, servers and connectivity healthy.	Sysadmin has access to servers and devices.	Sensors and backend are operational and any faults are resolved.

3.4 Relationships between use cases

- UC6 to UC11 each include UC12 (Log in), because dashboard access and configuration need authentication (REQ-DB-3, rule 5.5).
- UC8 (Modify route) extends UC7 (Generate optimized route).
- Marking a bin collected (UC4) also marks any linked complaint Resolved (business rule 5.5).
- External services that are not actors: the Mapping API supports UC7 and UC10, and Firebase Cloud Messaging delivers the UC5 alerts.
- The SRS names a system administrator but gives it no functional requirements, so UC15 is inferred from the user-class description in SRS 2.3.

4. Architectural design

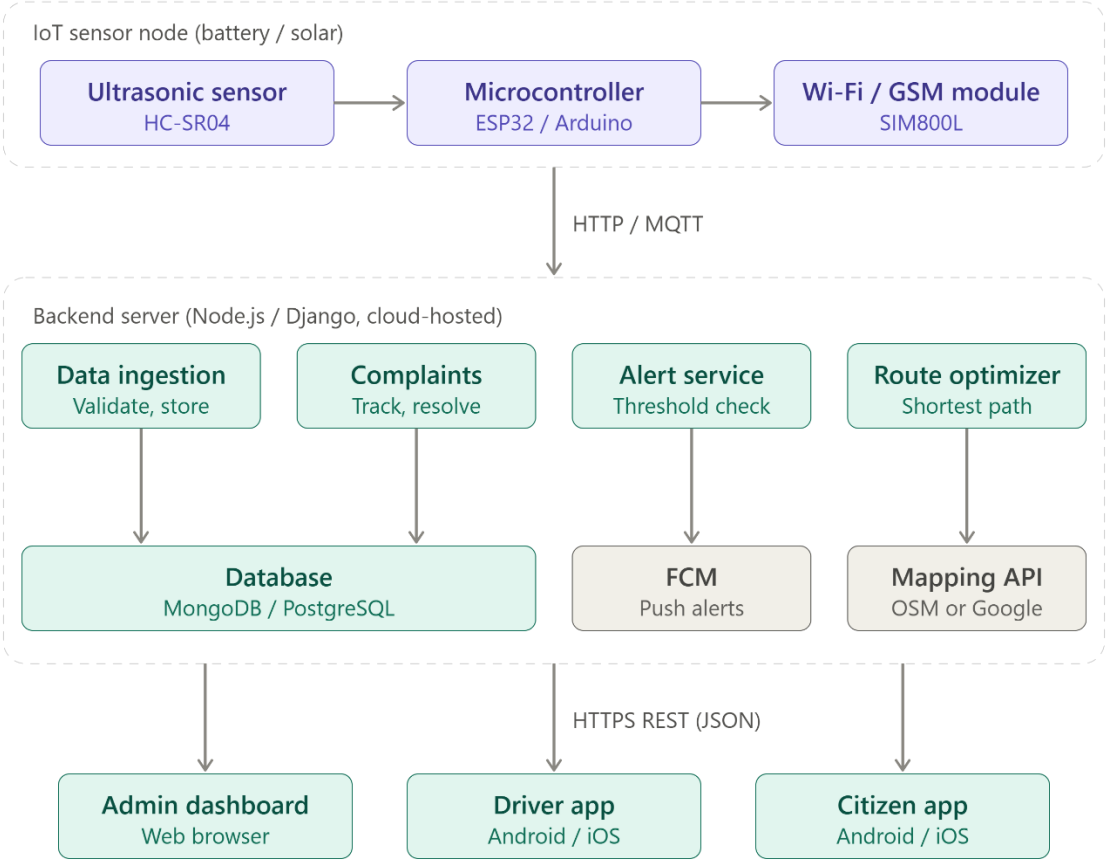
SWCMS uses a three-tier architecture: sensor nodes on the bins, a cloud backend, and three client apps. Sensors push data to the backend; clients pull from it over a REST API.

Device tier. An HC-SR04 ultrasonic sensor, an ESP32 or Arduino microcontroller and a Wi-Fi/GSM module (for example SIM800L), powered by battery or solar. Each node reports at a fixed interval, 15 minutes by default, over HTTP or MQTT.

Backend tier. A cloud-hosted Node.js or Django server on Linux. A REST API gateway fronts five feature modules (authentication, analytics, complaints, routing, alerting) and a separate ingestion service. Data lives in MongoDB or PostgreSQL. Firebase Cloud Messaging (FCM) and a mapping API sit behind adapters.

Client tier. An admin web dashboard for Chrome, Firefox and Edge, and driver and citizen mobile apps for Android 8.0+ and iOS 13+, built in Flutter or React Native. All clients use HTTPS REST with JSON.

4.1 System architecture diagram



The diagram shows sensor data entering through Ingestion, while all three clients go through the gateway. Colours mark the tiers; grey-outlined boxes are the database and third-party adapters.

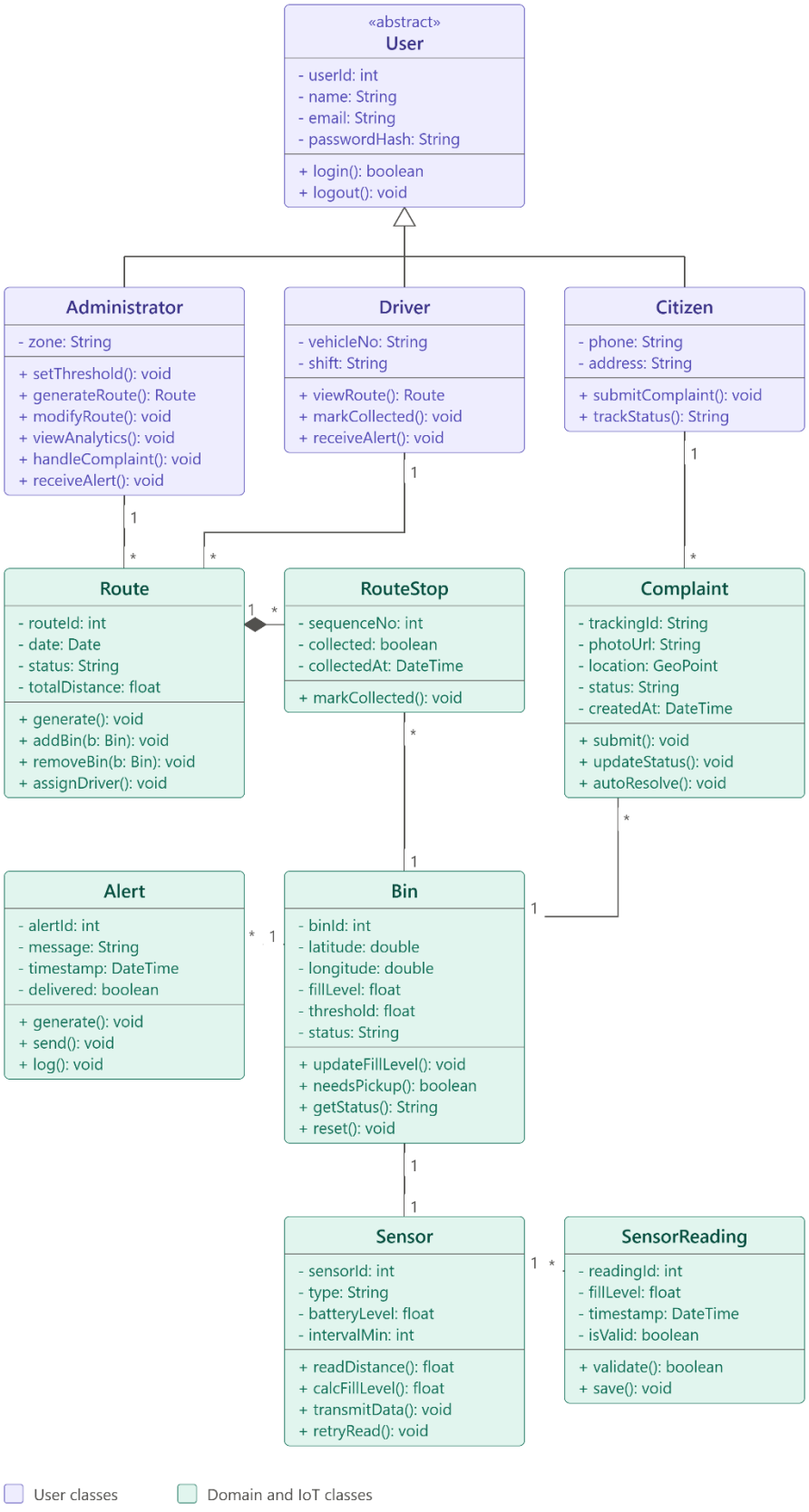
Pushing from the sensor at a fixed interval keeps the device logic small, which suits the low-power constraint. Keeping ingestion separate from the feature modules lets the high-volume sensor path scale without touching the rest.

5. Static design: class model

The class diagram identifies the main classes, their attributes and methods, and the relationships between them, taken from the SRS features and the use cases above.

5.1 UML class diagram

Eleven classes. User is abstract and is extended by Administrator and Driver; Citizen is a separate class because citizens have no account.


«abstract»
User
- userId: int
- name: String
- email: String
- passwordHash: String
+ login(): boolean
+ logout(): void
Administrator
- zone: String
+ setThreshold(): void
+ generateRoute(): Route
+ modifyRoute(): void
+ viewAnalytics(): void
+ handleComplaint(): void
+ receiveAlert(): void
Driver
- vehicleNo: String
- shift: String
+ viewRoute(): Route
+ markCollected(): void
+ receiveAlert(): void
Citizen
- phone: String
- address: String
+ submitComplaint(): void
+ trackStatus(): String
Route
- routeId: int
- date: Date
- status: String
- totalDistance: float
+ generate(): void
+ addBin(b: Bin): void
+ removeBin(b: Bin): void
+ assignDriver(): void
RouteStop
- sequenceNo: int
- collected: boolean
- collectedAt: DateTime
+ markCollected(): void
Complaint
- trackingId: String
- photoUrl: String
- location: GeoPoint
- status: String
- createdAt: DateTime
+ submit(): void
+ updateStatus(): void
+ autoResolve(): void
Alert
- alertId: int
- message: String
- timestamp: DateTime
- delivered: boolean
+ generate(): void
+ send(): void
+ log(): void
Bin
- binId: int
- latitude: double
- longitude: double
- fillLevel: float
- threshold: float
- status: String
+ updateFillLevel(): void
+ needsPickup(): boolean
+ getStatus(): String
+ reset(): void
Sensor
- sensorId: int
- type: String
- batteryLevel: float
- intervalMin: int
+ readDistance(): float
+ calcFillLevel(): float
+ transmitData(): void
+ retryRead(): void
SensorReading
- readingId: int
- fillLevel: float
- timestamp: DateTime
- isValid: boolean
+ validate(): boolean
+ save(): void
User classes
Domain and IoT classes

5.2 Relationships

Relationship	Type	Multiplicity	Meaning
Administrator, Driver to User	Generalization	n/a	Both inherit the shared account and login behaviour from the abstract User.
Administrator to Route	Association (generates)	1 to many	An admin generates many routes (REQ-RO-1, RO-3).
Driver to Route	Association (is assigned)	1 to many	A driver follows the route assigned each day.
Route to RouteStop	Composition	1 to many	A route is made of ordered stops; a stop cannot exist without its route.
RouteStop to Bin	Association (visits)	many to 1	Each stop points to one bin. A bin can appear on many routes over time.
Citizen to Complaint	Association (submits)	1 to many	A citizen can file many complaints (REQ-CF-1).
Complaint to Bin	Association (reports on)	many to 1	Many complaints can concern the same bin.
Alert to Bin	Association (raised for)	many to 1	A bin can trigger many alerts over time (REQ-AL-3).
Bin to Sensor	Association (monitored by)	1 to 1	Each automated bin carries one sensor node.
Sensor to SensorReading	Association (produces)	1 to many	Each sensor produces a time series of readings (REQ-BM-1, BM-2).

5.3 Class design decisions

- **RouteStop.** A route and a bin have a many-to-many relationship, and the collected status belongs to the pair, not to either side. RouteStop holds the sequence number, the collected flag and the collection time. When a driver marks a stop collected, the bin is reset and Complaint.autoResolve() then closes any linked complaint (rule 5.5).
- **Threshold logic.** Bin.needsPickup() compares fill level with the bin's threshold, and Route.generate() includes only bins where it is true. The threshold is stored per bin, so a global threshold (REQ-AL-1) is applied by updating every bin.
- **Alert delivery.** Alert.send() pushes through FCM to the admin and driver apps, which receive it in receiveAlert().
- **Sensor behaviour.** Sensor.retryRead() covers REQ-BM-3. SensorReading.isValid flags failed or out-of-range readings.

- **Analytics.** Administrator.viewAnalytics() builds the REQ-DB-2 report on demand from SensorReading and RouteStop data, so no separate report class is needed.
- **Accounts.** Administrators and drivers log in as Users. Citizens do not have accounts; they submit and track complaints with a tracking ID, as the SRS describes.

5.4 Classes and the components that own them

Component	Classes it owns
Auth service	User, Administrator, Driver
Ingestion service	Bin, Sensor, SensorReading
Routing	Route, RouteStop
Alerting	Alert
Complaints	Complaint, Citizen contact details

6. Behavioral design

Four UML views describe how the system behaves. They agree with each other and with the SRS; the findings below are what each view adds.

View	What it shows	Main finding
Activity (swimlane)	Four lanes: sensor node, backend server, admin/driver, citizen	Two flows start in parallel and meet at admin review
Sequence	Five participants over four phases	Alerts are pushed; every client request is request/response
Collaboration	Eight objects, ten numbered messages	The Complaints service is the busiest hub
State transition	Bin and complaint lifecycles	Bin states cycle; complaint states end at Resolved

Activity view. The sensor lane loops every 15 minutes: measure, compute fill percentage, transmit. The server stores the reading and tests it against the threshold. Below threshold, the sensor waits and repeats. At or above, the server raises an alert. A citizen complaint starts a second flow in parallel. Alerts and complaints both end at one admin review step, which keeps the dashboard as the single place where staff decide what to collect. The flow continues through route request, optional route adjustment, driver collection, and a server update that closes related complaints.

Sequence view. The sequence view groups messages into a 15-minute loop with a threshold alt-frame, a citizen complaint, route optimization and collection. Reading uploads and complaint

submission are acknowledged; alerts and route pushes are one-way notifications. The view shows that the dashboard never talks to the sensor, so all threshold checks happen on the server.

Collaboration view. The Complaints service handles four of the ten messages: submit, forward, mark collected and resolved. The Alert service handles three and the Route service two. That makes Complaints the first module to watch for coupling. One design gap shows here: the driver's mark-collected message goes to Complaints, but a bin's collected status is bin data, not complaint data. The class model resolves this. Routing records the collection on the route stop (collected flag and time), the bin record is reset, and Complaints then auto-resolves any linked complaints. Complaints no longer receives the driver's message directly, which also lowers its coupling.

State transition view. A bin moves Normal, Alert, Route assigned, Collected, then back to Normal on the next reading below threshold. An admin removing a bin from a route returns it to Alert. A complaint is Pending until its bin is recorded as collected, then Resolved automatically (business rule 5.5). Sensor read errors loop back with retry and logging (REQ-BM-3).

7. Component design

The system splits into 14 components. Each backend module has one job and talks to the others only through the gateway or the database, which meets the modularity requirement in SRS 5.4.

Component	Responsibility	Uses
Sensor firmware	Read distance, compute fill %, transmit with bin ID and timestamp, retry on error	Ingestion service
Ingestion service	Validate and store readings	Database
REST API gateway	Single HTTPS entry point for all clients; request routing	All backend modules
Auth service	Login for admins (optional OTP) and drivers; guards admin-only actions. Citizens submit and track complaints with a tracking ID and no account.	Database
Analytics	Collection counts and average fill-level per bin over a date range	Database
Complaints	Create complaints, assign tracking IDs, track status, auto-resolve when Routing reports a bin collected	Database
Routing	Select bins at or above threshold, compute route, accept manual changes,	Maps adapter, Database

Component	Responsibility	Uses
	record collected stops	
Alerting	Check threshold per bin or globally, raise and log alerts	FCM adapter
Maps adapter	Hides the mapping provider (OpenStreetMap or Google Maps)	External API
FCM adapter	Sends push notifications to admin and driver apps	Firebase Cloud Messaging
Database	Bin data, sensor logs, users, alerts, complaints	None
Admin dashboard	Colour-coded map, alert list, route view, reports	Gateway
Driver app	Daily route, mark bin as collected	Gateway
Citizen app	Submit complaints, view complaint status	Gateway

The component diagram leaves out the Routing and Alerting reads from the database to stay readable. In implementation both read bin fill-levels and thresholds from it.

8. Requirements traceability

All 15 functional requirements map to at least one component, one diagram and one use case. The last column shows where the SRS and the design need a closer look.

Requirement	Summary	Design element	Shown in	Note
REQ-BM-1	Measure at a configurable interval, default 15 min	Sensor firmware	Activity, sequence, UC13	Interval config path not yet designed
REQ-BM-2	Send level with bin ID and timestamp	Sensor firmware, Ingestion	Sequence, UC14	Covered
REQ-BM-3	Retry and log failed reads	Sensor firmware, Database	State, UC13	Where the log is kept is a design choice
REQ-AL-1	Configurable threshold per bin or globally	Alerting, Auth, Database	Component, UC9	Admin-only per rule 5.5

Requirement	Summary	Design element	Shown in	Note
REQ-AL-2	Alert within 5 min of crossing threshold	Alerting, FCM adapter	Sequence, state, UC5	See risk R1 in section 11
REQ-AL-3	Log alerts with bin ID and timestamp	Alerting, Database	Component, class Alert	Covered
REQ-RO-1	Route covers all bins above threshold	Routing	Activity, sequence, UC7	Covered
REQ-RO-2	Route in 10 s for up to 200 bins	Routing, Maps adapter	Component, UC7	Depends on TBD-1
REQ-RO-3	Admin adds or removes a bin	Admin dashboard, Routing	Activity, state, UC8	Covered
REQ-DB-1	Colour-coded bin map	Admin dashboard	Component, UC10	Green, yellow, red bands not defined
REQ-DB-2	Collections and average fill report	Analytics	Component, UC11	Covered
REQ-DB-3	Admin-only dashboard access	Auth service, gateway	Component, UC12	Covered
REQ-CF-1	Submit complaint with location and photo	Citizen app, Complaints	Activity, sequence, UC1	Photo storage not designed
REQ-CF-2	Unique tracking ID	Complaints	Collaboration, sequence, class Complaint	Covered
REQ-CF-3	View Pending or Resolved status	Citizen app, Complaints	State, UC2	Not shown in activity or sequence

9. Design decisions and trade-offs

The SRS leaves three decisions open (Appendix C) and offers several either/or technology choices. The table gives a recommendation for each, so the team can confirm or change it.

Decision	Options	Recommendation	Reason
TBD-1: route	Nearest-neighbor heuristic or	Start with nearest-neighbor behind a	Simple, fast for 200 bins, and swappable if route quality needs

Decision	Options	Recommendation	Reason
algorithm	Vehicle Routing Problem solver	Routing interface	improving
TBD-2: mapping provider	Google Maps or OpenStreetMap	OpenStreetMap	SRS 2.5 asks for open or low-cost APIs; the Maps adapter keeps a switch cheap
TBD-3: default threshold	Value set after pilot	Keep configurable; use 80% as the working value	SRS 4.2.2 uses 80% as its example; pilot data will confirm it
Sensor transport	HTTP or MQTT	HTTP first, MQTT if GPRS cost or bandwidth bites	HTTP is simpler to prototype; MQTT is lighter on constrained links
Database	MongoDB or PostgreSQL	PostgreSQL	Bins, routes and complaints are related records, and REQ-DB-2 needs date-range reports
Backend	Node.js or Django	Team's choice	Both satisfy the REST design; pick the one the team knows best
Mobile framework	Flutter or React Native	Team's choice	Both cover Android 8.0+ and iOS 13+

These are design recommendations, not SRS requirements. The first three should be recorded in the SRS once agreed, replacing the TBD entries.

10. Nonfunctional analysis

The design meets the nonfunctional requirements on paper; the last column lists what to check on the prototype.

Area	SRS target	How the design meets it	Check in testing
Performance: ingestion	500 concurrent sensor nodes	At 15-minute intervals, 500 nodes send about 0.56 readings per second on average; ingestion is its own service	Burst load if nodes sync up; add random start offsets
Performance: dashboard	Loads in 3 s	Gateway returns bin status in one JSON call	Measure with 500 bins on the map
Performance: routing	10 s for 200 bins	Heuristic solver in a dedicated module	Time it at 200 bins
Security	TLS,	HTTPS on every link; Auth service	Check that admin

Area	SRS target	How the design meets it	Check in testing
	authenticated admins, protected citizen data	with optional OTP; citizen data used only for complaints	endpoints reject unauthenticated calls
Reliability	95% of transmissions succeed	Retry and log on failed reads and sends	Measure on real GPRS coverage
Scalability	More bins and zones without redesign	Stateless modules behind a gateway; ingestion scales alone	Add a second zone in the pilot
Maintainability	Modular codebase	One module per feature; adapters isolate third parties	Code review against module boundaries
Safety	Never block manual collection	The system only recommends routes and sends alerts; it has no control over collection itself	Confirm crews can work without the app

Weatherproofing and electrical enclosure for the sensor hardware are hardware tasks outside this software design.

11. Risks and limitations

The biggest risk is the alert timing in REQ-AL-2, which the 15-minute reporting interval can undermine.

ID	Risk	Impact	Mitigation
R1	REQ-AL-2 asks for an alert within 5 minutes of crossing the threshold, but readings arrive every 15 minutes	A bin can be full for up to 15 minutes before the system knows	Measure the 5 minutes from the server receiving the reading, and record that in the SRS
R2	Weak GPRS coverage at some bins	Missed readings, lower than 95% success	Retry and log; pick pilot bins with stable coverage (SRS 2.7)
R3	Battery or solar power runs short	Sensor goes silent	Alert admins when a bin has not reported for a set number of intervals
R4	Ultrasonic reading distorted by uneven waste or weather	Wrong fill percentage	Calibrate per bin; log failed reads (REQ-BM-3)

ID	Risk	Impact	Mitigation
R5	Mapping API cost or limits	Routing fails or becomes costly	Maps adapter plus OpenStreetMap default
R6	Citizen location and contact data	Privacy exposure	Store securely and use only for complaint resolution (SRS 5.3)
R7	Driver marks bins collected only when online	Complaints stay Pending	Queue the update on the phone and send when the connection returns

The mitigations for R3, R4 and R7 are design suggestions beyond the SRS; the team can add them as requirements if agreed.

12. Conclusion and next steps

The design covers all 15 functional requirements and the main nonfunctional targets, using a three-tier architecture with a modular backend. The use case and class models tie the actors, features and data together, and the class model closes the earlier gap over who owns a bin's collected status. Its weakest points are the 5-minute alert target, the three open TBD items and a few small design gaps.

Next steps:

- Settle TBD-1, TBD-2 and TBD-3 and update SRS Appendix C
- Clarify the starting point of the 5-minute alert time in REQ-AL-2
- Confirm that Routing owns a bin's collected status, as in the class model, and update the collaboration diagram to match
- Add driver accounts to the SRS (section 5.3), since routes and alerts are sent to a specific driver
- Define the green, yellow and red bands for the dashboard map
- Add a data-flow diagram and a database design to SRS Appendix B
- Test the performance checks in section 10 on the prototype